

Mission 3 – Elite Assessment: Agentic AI System, Runtime, Tooling, and Evaluation

Ready-to-send technical assignment for evaluating candidates on agentic AI design, implementation, runtime orchestration, tool selection, and evaluation discipline.

1. Objective

Design and build a small but real agentic AI system that can plan, use tools, coordinate multiple steps, and produce a grounded business recommendation.

This assignment is specifically designed to test:

- ability to design agent workflows
- ability to choose the right orchestration/runtime framework
- ability to use tools effectively
- ability to evaluate agent quality, not just demo a happy path
- ability to explain tradeoffs clearly

2. What We Want to Assess

We are not looking for a basic chatbot.

We are looking for evidence that you can:

- design a true agentic workflow
- decide when to use tools vs prompts vs deterministic logic
- choose a framework/runtime intentionally
- evaluate reliability, grounding, and task success
- think like a production engineer, not only a demo builder

3. Scope (What You Must Build)

You must build an agentic AI system that:

- accepts a user request
- decides what steps are needed
- uses one or more tools
- retrieves and reasons over provided knowledge
- produces a structured recommendation
- returns an execution trace
- includes an evaluation method for measuring quality

Minimum required capabilities:

1. Planner / Orchestrator

Decides which actions to take and in what order.

2. Retrieval Tool

Retrieves relevant information from the provided sample dataset / knowledge base.

3. Business Logic Tool or Scoring Tool

Applies deterministic logic or scoring rules.

4. Response / Decision Agent

Produces the final structured answer.

5. Evaluation Flow

Measures task success and identifies failures.

Important:

Your system must show actual tool use and orchestration.

One large prompt pretending to be agentic behavior is not acceptable.

4. Mandatory Agentic AI Focus

Your solution must explicitly demonstrate the following:

- planning or task decomposition
- tool invocation
- state handling across steps

- structured intermediate outputs
- controlled final answer synthesis
- basic failure handling / fallback

Strong solutions will also show:

- retry or repair behavior
- confidence handling
- separation between reasoning and action
- guardrails for unsupported or low-confidence answers

5. Sample Dataset (Use This in Your Solution)

Use the following embedded sample dataset. You may extend it, but your solution must work correctly on this base set.

A. Customer Profiles

1. Customer A

Industry: Healthcare

Region: Riyadh

Employees: 120

Dependents Ratio: 0.60

Budget: Medium

Priority: Balanced coverage and network strength

2. Customer B

Industry: Construction

Region: Jeddah

Employees: 80

Dependents Ratio: 0.30

Budget: Low

Priority: Cheapest acceptable option

3. Customer C

Industry: Retail

Region: Dammam

Employees: 200

Dependents Ratio: 0.50

Budget: Medium

Priority: Stable service and moderate cost

B. Package Catalog

1. Basic

Network: C

Price Range: 4000–5000

Coverage: Low

Notes: Cheapest option, limited provider access

2. Standard

Network: B

Price Range: 6000–7500

Coverage: Medium

Notes: Balanced price and network

3. Premium

Network: A

Price Range: 9000–12000

Coverage: High

Notes: Best coverage and strongest network

C. Benchmark Rules

- Healthcare = high risk
- Construction = medium risk

- Retail = medium-low risk
- High dependents ratio increases cost pressure
- Riyadh has higher medical cost than Jeddah and Dammam
- Jeddah is moderate cost
- Dammam is moderate-to-high cost
- Low-budget users should not be recommended Premium unless clearly justified
- If user asks for “best coverage”, favor Premium unless budget or rules strongly conflict
- If user asks for “cheapest acceptable”, favor Basic or Standard with explanation

D. Optional Knowledge Snippets

- Network A is best for customers who prioritize access and premium hospitals.
- Network B is best for balanced price-to-value tradeoffs.
- Network C is suitable for cost-focused customers with simpler expectations.
- High-dependents healthcare accounts tend to need tighter control of benefits and cost tradeoffs.

6. What the System Should Do

Your system should answer requests such as:

- Recommend the best plan for a healthcare company in Riyadh
- Give me the cheapest acceptable option for a construction customer in Jeddah
- Compare Standard vs Premium for a retail customer
- Explain why this recommendation was made
- Ask for clarification when the request is incomplete or ambiguous

The system should produce:

- recommendation
- reasoning
- tools used
- confidence

- execution trace

7. Required Architecture Components

Your submission must include these components:

A. Planner / Orchestrator

Responsible for:

- understanding the request
- deciding which tools/agents to call
- managing state
- aggregating final output

B. Tools

At minimum:

- retrieval tool
- scoring / rules tool

You may add:

- comparison tool
- validation tool
- fallback / escalation tool

C. Output Composer

Produces final structured answer.

D. Evaluation Layer

Assesses whether the agent completed the task correctly and reliably.

8. Tooling and Runtime Selection (VERY IMPORTANT)

You must choose and justify the tools/frameworks used to build, run, and evaluate the agent system.

A. Build / Orchestration Framework

Choose one or more of:

- LangGraph
- Semantic Kernel
- AutoGen
- CrewAI
- OpenAI SDK / custom orchestration
- Azure AI Agent Service / Microsoft stack equivalent
- Other framework if justified

You must explain:

- why you selected it
- why it fits this problem
- why you did not choose other options
- what tradeoffs it creates

B. Tool Runtime / Agent Execution

Explain how agents are executed:

- sequential
- graph-based
- planner-executor
- event-driven
- stateful workflow

C. Evaluation / Observability Tools

Choose one or more of:

- Langfuse
- Promptfoo
- MLflow
- custom evaluation harness
- OpenTelemetry / tracing

- Azure monitoring or equivalent

You must explain:

- how you would trace agent steps
- how you would measure success
- how you would detect hallucination or failure

9. Model Selection Criteria (Mandatory)

You must explicitly justify model selection.

Your answer must explain:

- which LLM(s) you selected
- where each model is used
- why that model is appropriate
- tradeoffs: reasoning quality, cost, latency, context length, multilingual support

Examples of acceptable model strategy:

- strong reasoning model for planner/orchestrator
- smaller or cheaper model for extraction/routing
- deterministic rules tool instead of LLM where appropriate

You do not have to use multiple models, but if you use one model everywhere you must explain why that is the correct decision.

10. Required Output Format (Strict)

Your system must return output in a structured format similar to this:

```
{  
  "user_request": "...",  
  "plan": {  
    "steps": [  
      "retrieve relevant package knowledge",  
      "apply scoring rules",
```

```
    "compose final recommendation"
  ]
},
"tools_used": [
  "retrieval_tool",
  "pricing_or_scoring_tool"
],
"recommendation": {
  "plan_name": "...",
  "network": "...",
  "price_range": [min, max]
},
"reasoning": [
  "driver 1",
  "driver 2",
  "driver 3"
],
"confidence": "...",
"execution_trace": [
  "planner selected retrieval tool",
  "retrieval returned Standard and Premium",
  "scoring tool penalized Premium due to budget",
  "final answer selected Standard"
],
"fallback_or_risk_note": "..."
}
```

Free-text only answers are not acceptable.

11. Test Scenarios (Visible Evaluation Cases)

We will test the system with at least the following visible scenarios:

Scenario 1 – Balanced recommendation

Input:

“Recommend the best plan for a healthcare company in Riyadh.”

Expected:

- retrieval + scoring or planning
- Standard or Premium, but must be justified
- clear explanation of risk, region, and dependents impact

Scenario 2 – Cost-focused recommendation

Input:

“Give me the cheapest acceptable option for a construction customer in Jeddah.”

Expected:

- Basic or Standard
- strong cost reasoning
- tradeoff explanation

Scenario 3 – Comparison workflow

Input:

“Compare Standard and Premium for a retail customer in Dammam.”

Expected:

- comparison path
- explicit pros/cons
- recommendation based on stated priorities

Scenario 4 – Missing information

Input:

“Recommend a plan for my company.”

Expected:

- asks for clarification OR uses safe fallback logic
- does not hallucinate details

Scenario 5 – Explanation request

Input:

“Why did you choose that recommendation?”

Expected:

- reason trace
- tool trace
- not just generic explanation

12. Hidden Edge Cases (We Will Use Secretly)

We will also test hidden cases such as:

- conflicting constraints: low budget + best coverage
- unsupported question outside dataset
- intentionally ambiguous wording
- incomplete data fields
- retrieval returns partially relevant results
- tool failure or empty result
- model overconfidence
- planner chooses unnecessary steps
- repeated tool calls without improvement

Your system should demonstrate graceful failure, not just happy-path success.

13. Deliverables

You must submit:

1. Working code

2. README with setup instructions
3. Short technical report (max 5 pages)
4. Architecture diagram
5. Explanation of tool/framework/model choices
6. Evaluation report with at least 5 evaluated runs
7. Demo walkthrough (video or live)

Your technical report must cover:

- system architecture
- agent/runtime design
- tool selection
- model selection
- evaluation strategy
- limitations and next steps

14. Evaluation Scorecard

We will score the solution as follows:

1. Agentic Architecture – 25%
 - real orchestrator/planner
 - clear use of tools
 - modular workflow
 - state handling
2. Tooling and Runtime Decisions – 20%
 - good framework choice
 - runtime clarity
 - intentional orchestration design
3. Evaluation and Reliability – 20%
 - measurable quality

- visible test harness

- failure detection

- observability/tracing

4. Model Selection and Tradeoff Thinking – 15%

- model justified well

- cost/performance awareness

- clear role per model

5. Implementation Quality – 10%

- code quality

- structure

- reproducibility

6. Communication Quality – 10%

- clarity

- explanation

- design defense

15. Junior Version (Reduced Expectation Option)

For a junior-level candidate, the assignment can be reduced to:

- 1 orchestrator

- 2 tools

- no advanced evaluation stack required

- basic trace logging

- single-model setup allowed

- no retry/repair behavior required

Junior candidates are still expected to:

- show real tool use

- return structured output

- explain their design choices

16. What a Perfect Solution Looks Like

A top-tier solution will usually have these characteristics:

- graph-based or stateful orchestration
- clear distinction between planner, tools, and final answer generator
- at least one deterministic tool for scoring or business logic
- structured intermediate steps
- visible execution trace
- good framework selection and explanation
- thoughtful model selection
- lightweight but real evaluation harness
- graceful fallback when data is missing
- no hallucinated certainty
- clean code and clear documentation

We are not looking for maximum complexity.

We are looking for strong judgment, clarity, and production-style thinking.

17. Red Flags (Automatic Rejection Signals)

The following are strong negative signals:

- one large prompt presented as “multi-agent”
- no real tool execution
- no orchestrator logic
- no evaluation layer
- no model/framework justification
- no structured outputs
- hard-coded answers disguised as AI
- overengineered architecture with weak execution

- strong claims with no evidence

18. Sample Output Scenario (Reference Example)

Example input:

“Give me the cheapest acceptable option for a construction customer in Jeddah.”

Example acceptable output:

```
{  
  "user_request": "Give me the cheapest acceptable option for a construction customer in Jeddah.",  
  "plan": {  
    "steps": [  
      "retrieve relevant package options",  
      "apply cost and risk rules",  
      "compose recommendation"  
    ],  
  },  
  "tools_used": [  
    "retrieval_tool",  
    "scoring_tool"  
  ],  
  "recommendation": {  
    "plan_name": "Basic",  
    "network": "C",  
    "price_range": [4000, 5000]  
  },  
  "reasoning": [  
    "Construction is medium risk, not high enough to force Premium",
```

```

    "User priority is cheapest acceptable option",
    "Jeddah has moderate cost pressure, so Basic remains viable"
  ],
  "confidence": "medium-high",
  "execution_trace": [
    "Planner selected retrieval tool",
    "Retrieval returned Basic and Standard as relevant options",
    "Scoring tool ranked Basic highest for cost priority",
    "Final response generated with tradeoff explanation"
  ],
  "fallback_or_risk_note": "Basic is the lowest-cost option but may provide weaker network access than Standard."
}

```

This is only a reference style. Your exact values may differ if justified clearly.

19. Final Note to Candidate

We care more about correctness, judgment, and system thinking than visual polish.

If you make assumptions, state them.

If your system cannot answer confidently, show how it fails safely.

The best solutions are usually disciplined, explainable, and measurable.